```
sdk install java 25-open
```

```
Problemas  Salida  Consola de depuración  Terminal  Puertos 4

@Mxteo2008 ➜/workspaces/Talleres (main) $ sdk install java 25-open
Installing: java 25-open
Done installing!


Setting java 25-open as default.
@Mxteo2008 ➜/workspaces/Talleres (main) $
```

JSHELL

```
@Mxteo2008 ➜/workspaces/Talleres (main) $ sdk install java 25-open
Setting java 25-open as default.
@Mxteo2008 ➜/workspaces/Talleres (main) $ jshell
|  Welcome to JShell -- Version 11.0.32.1
|  For an introduction type: /help intro

jshell>
```

```
Problemas  Salida  Consola de depuración  Terminal  Puertos 5

@Mxteo2008 ➜/workspaces/Talleres (main) $ jshell

jshell> var nums = List.of(1, 2, 3, 4, 5);
   ...> nums.stream().map(n -> n * 10).toList();
   ...> record Tuple(int x, int y){};
   ...> nums.stream().map(n -> new Tuple(n, n * n)).toList();
   ...> nums.stream().filter(n -> n % 2 == 0).toList();
   ...> nums.stream().reduce(1, (n1, n2) -> n1 * n2);
   ...>
nums ==> [1, 2, 3, 4, 5]
$3 ==> [10, 20, 30, 40, 50]
|  created record Tuple
$6 ==> [Tuple[x=1, y=1], Tuple[x=2, y=4], Tuple[x=3, y=9], Tuple[x=4, y=16], Tuple[x=5, y=25]]
$7 ==> [2, 4]
$8 ==> 120

jshell>
```

Scala

```
Para Java (jshell)

Problemas  Salida  Consola de depuración  Terminal  Puertos 5

@Mxteo2008 ➜/workspaces/Talleres (main) $ scala
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

Welcome to Scala 3.3.1 (25, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala>
```

```
@Mxteo2008 ➜/workspaces/Talleres (main) $ scala
Welcome to Scala 3.3.1 (25, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> val nums = List(1, 2, 3, 4, 5)
     | nums.map(_ * 10)
     | nums.map(n => (n, n * n))
     | nums.filter(n => n % 2 == 0)
     | nums.reduce((n1, n2) => n1 * n2)
     |
val nums: List[Int] = List(1, 2, 3, 4, 5)
val res0: List[Int] = List(10, 20, 30, 40, 50)
val res1: List[(Int, Int)] = List((1,1), (2,4), (3,9), (4,16), (5,25))
val res2: List[Int] = List(2, 4)
val res3: Int = 120

scala>
```

Python

```
@Mxteo2008 ➜/workspaces/Talleres (main) $ ipython
Python 3.10.22 (main, Oct  8 2026, 16:42:09) [GCC 11.4.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.39.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

```
Problemas  Salida  Consola de depuración  Terminal  Puertos 5

@Mxteo2008 ➜/workspaces/Talleres (main) $ ipython
Python 3.10.22 (main, Oct  8 2026, 16:42:09) [GCC 11.4.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.39.0 -- An enhanced Interactive Python. Type '?' for help.
Ctrl click to launch VS Code Native REPL (https://aka.ms/python-native-repl)

In [1]: lst = [1, 2, 3, 4, 5]
   ...: list(map(lambda n: n * 10, lst))
   ...: list(map(lambda n: (n, n * n), lst))
   ...: list(filter(lambda n : n % 2 == 0, lst))
   ...: from functools import reduce
   ...: reduce(lambda x, y : x * y, lst, 1)
   ...:
Out[1]: 120
```

TRANSFORMAR DE JAVA A SCALA

```java
double getAverage(List<Integer> nums) {
    var sum = 0;
    for(var num : nums) {
        sum += num;
    }
    return sum / (double)nums.size();
}
```

EN SCALA:

```scala
def getAverage(nums: List[Int]): Double =
  nums.sum.toDouble / nums.size
```

HASKELL

```haskell
getAverage :: [Int] -> Double
getAverage nums = fromIntegral (sum nums) / fromIntegral (length nums)
```

PYTHON

```python
def get_average(nums: list[int]) -> float:
    return sum(nums) / len(nums)
```

SCALA

```scala
// Guardar este archivo como: average.scala
def getAverage(nums: List[Int]): Double =
  nums.sum.toDouble / nums.size

@main def principal(): Unit =
  val numeros = List(1, 2, 3, 4, 5)
  println(s"El promedio es: ${getAverage(numeros)}")
```